

Axum

I. Lesson

Backend frameworks

Axum

Actix

Rocket

Backend frameworks

Feature	Axum	Actix Web	Rocket
Async Runtime	Tokio	Actix (built on Tokio)	Tokio (since v0.5)
Performance	Excellent (slightly behind Actix)	Top-tier (fastest in benchmarks)	Good, but not as fast as others
Ergonomics	Clean, modular, functional style	Low-level but powerful	Very ergonomic, macro-based
Learning Curve	Moderate to steep (functional model)	Steep (actor model, lower-level)	Easy to moderate (developer-friendly)
Community & Ecosystem	Growing fast, part of Tokio	Mature, performance-focused users	Popular among beginners, friendly docs

1. Rocket



```
#[macro_use] extern crate rocket;

#[get("/hello")]
fn hello() -> &'static str {
    "Hello, world!"
}

#[launch]
fn rocket() -> _ {
    rocket::build().mount("/", routes![hello])
}
```

2. Axum

```
● ● ●  
  
use axum::{Router, routing::get, response::IntoResponse};  
use std::net::SocketAddr;  
  
async fn hello() -> impl IntoResponse {  
    "Hello, world!"  
}  
  
#[tokio::main]  
async fn main() {  
    let app = Router::new().route("/hello", get(hello));  
  
    let addr = SocketAddr::from(([127, 0, 0, 1], 3000));  
    axum::Server::bind(&addr)  
        .serve(app.into_make_service())  
        .await  
        .unwrap();  
}
```

3. Actix

```
● ● ●  
  
use actix_web::{get, App, HttpServer, Responder};  
  
#[get("/hello")]  
async fn hello() -> impl Responder {  
    "Hello, world!"  
}  
  
#[actix_web::main]  
async fn main() -> std::io::Result<()> {  
    HttpServer::new(|| {  
        App::new().service(hello)  
    })  
    .bind(("127.0.0.1", 3000))?  
    .run()  
    .await  
}
```

What makes Axum special? 🤔

Built on top of the **Tokio** async runtime and **Tower** middleware stack.

Axum avoids macros and magic, favoring explicit, composable code that scales well for complex APIs and microservices.

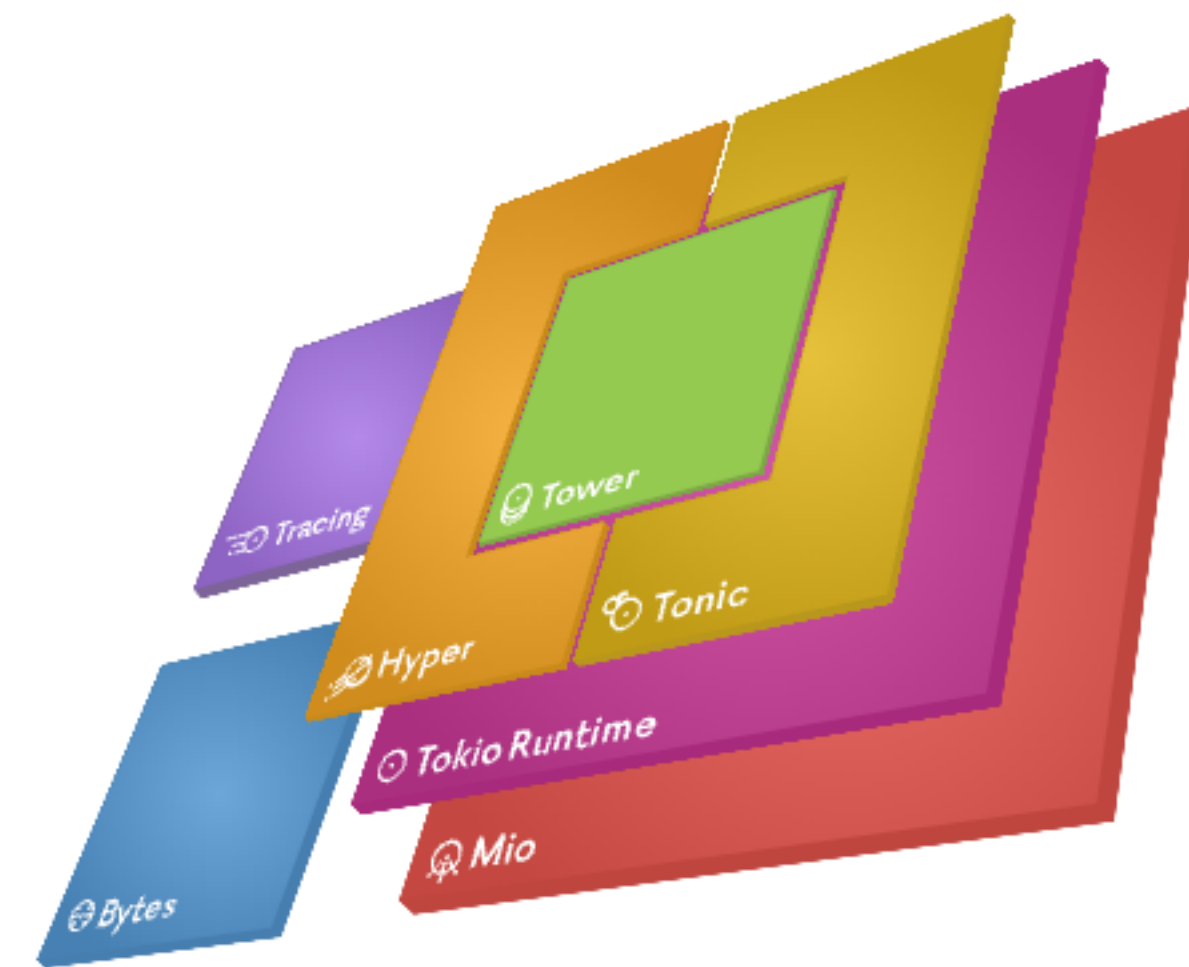
It also integrates well with Fullstack frameworks like Leptos.

The Tokio stack



Applications aren't built in a vacuum. The Tokio stack includes everything needed to ship to production, fast.

Stack
Runtime
Hyper
Tonic
Tower
Mio
Tracing
Bytes



⦿ Runtime

Including I/O, timer, filesystem, synchronization, and scheduling facilities, the Tokio runtime is the foundation of asynchronous applications.

[Learn more →](#)

Benefits of Tokio

With **Tokio**, you can write **async code easily in Rust**, which means your program can handle many tasks at once without blocking. It takes care of running your async functions, scheduling them efficiently,

Axum Router

The **Axum router** is the core of how you define and organize your web app's routes in Axum. It lets you map URLs like */hello* or */users/{id}* to specific handler functions.

It's very flexible and easy to build with—you can chain routes together, nest them, or apply middleware to groups of routes.

II. Project

Simple Todo App

Todo struct



```
#[derive(Debug, Serialize, Clone)]
pub struct Todo {
    pub id: i32,
    pub text: String,
}
```

The serde macro



```
use serde::{Serialize, Deserialize};
```

Impl handlers for Todo



```
impl Todo {  
    pub async fn get_all(...)   
  
    pub async fn get_by_id(...)   
  
    pub async fn create(...)   
  
    pub async fn update(...)   
  
    pub async fn delete(...)   
}
```

Handler

A **handler** is a function that process a **request** and returns a **response**.

Routes



```
pub fn todo_routes() -> Router<DbArcRwLock> {  
    Router::new()  
        .route("/todos", get(Todo::get_all).post(Todo::create))  
        .route(  
            "/todos/{id}",  
            get(Todo::get_by_id)  
                .patch(Todo::update)  
                .delete(Todo::delete),  
        )  
}
```

Project: Simple Todo App with Axum

Your turn!